

Lucid Datenstrom

Datenstrom Background and Vision

- › Many projects exhibiting common *structured data access* challenges
- › Early proto designs primarily focused on binary protocols / files
- › Later proto designs primarily focused on RDBMS access
- › Over the years, designs evolved with the evolving C++ standard
- › These ideas were finally merged to produce Datenstrom
- › Initial git import – 12:37pm 1 June 2018
- › Current version 0.22.0-beta
- › Synergistically combines concepts, techniques and technologies
- › A combination whose value exceeds the sum of its parts
- › Work in progress with unfinalized and competing design concerns
- › However, these rough edges are becoming more streamlined

Datenstrom General Information

- › Datenstrom (Deutsch) – data stream
- › Multiplatform – vendor agnostic
- › C++17 – ISO/IEC 14882:2017
- › 32 / 64 bit clean
- › Strongly influenced by STL
- › Highly compatible with STL
- › Data encoding framework – fields and records
- › Delegated encoding – RDBMS abstraction layer
- › Bespoke encoding – CSV, binary etc...
- › Separation of concerns – per encoding class
- › Nuget package: Lucid.Datenstrom

Basic Abstractions – namespace ds

```
class istream
{
public:

    typedef istream & ( *manip_t )( istream & );

    virtual ~istream( void );

    virtual istream & operator >> ( int8_t & );
    virtual istream & operator >> ( int16_t & );
    virtual istream & operator >> ( int32_t & );
    virtual istream & operator >> ( int64_t & );

    virtual istream & operator >> ( uint8_t & );
    virtual istream & operator >> ( uint16_t & );
    virtual istream & operator >> ( uint32_t & );
    virtual istream & operator >> ( uint64_t & );

    virtual istream & operator >> ( double & );

    virtual istream & operator >> ( std::string & );

    virtual void read( void *, size_t );

    virtual void endr( void );

    virtual bool eof( void ) const;
};
```

```
class ostream
{
public:

    typedef ostream & ( *manip_t )( ostream & );

    virtual ~ostream( void );

    virtual ostream & operator << ( int8_t );
    virtual ostream & operator << ( int16_t );
    virtual ostream & operator << ( int32_t );
    virtual ostream & operator << ( int64_t );

    virtual ostream & operator << ( uint8_t );
    virtual ostream & operator << ( uint16_t );
    virtual ostream & operator << ( uint32_t );
    virtual ostream & operator << ( uint64_t );

    virtual ostream & operator << ( double );

    virtual ostream & operator << ( const char * );
    virtual ostream & operator << ( const std::string & );

    virtual void write( const void *, size_t );

    virtual void endr( void );
};
```

STL Comparison to Datenstrom – atomic data (field) separation

```
std::ostream out;
```

```
out << "Hello" << "World";
```

```
std::istream in;
```

```
std::string s1;
```

```
std::string s2;
```

```
in >> s1 >> s2;
```

```
assert( s1 == "HelloWorld" );
```

```
assert( s2 == "" );
```

```
ds::ostream out;
```

```
out << "Hello" << "World";
```

```
ds::istream in;
```

```
std::string s1;
```

```
std::string s2;
```

```
in >> s1 >> s2;
```

```
assert( s1 == "Hello" );
```

```
assert( s2 == "World" );
```

I/O Manipulators – namespace ds

```
class istream
{
public:

    typedef istream & ( *manip_t )( istream & );

    virtual void endr( void );
};

istream & endr( istream & in )
{
    in.endr();
    return in;
}

istream & operator >> istream & in, istream::mainip_t & fn )
{
    return fn( in );
}
```

```
class ostream
{
public:

    typedef ostream & ( *manip_t )( ostream & );

    virtual void endr( void );
};

ostream & endr( ostream & out )
{
    out.endr();
    return out;
}

ostream & operator << ( ostream & out, ostream::manip_t & fn )
{
    return fn( out );
}
```

Datenstrom – data collection (record) separation

```
ds::ostream out;
```

```
out << "Hello" << "World" << ds::endr  
    << "Goodbye" << "World" << ds::endr;
```

```
ds::istream in;
```

```
std::string s1;  
std::string s2;
```

```
in >> s1 >> ds::endr  
    >> s2 >> ds::endr;
```

```
assert( s1 == "Hello" );  
assert( s2 == "Goodbye" );
```

```
ds::istream in;
```

```
std::string s1;  
std::string s2;  
std::string s3;
```

```
in >> s1 >> s2 >> s3 >> ds::endr;
```

```
// Exception
```

Custom and Aggregate type integration - definitions

```
enum class job_type_t { floor = 'F', wall = 'W', roof = 'R', general = 'G' };
```

```
struct job_key_t  
{  
    std::string id;  
    job_type_t type;  
};
```

```
struct job_t  
{  
    job_key_t key;  
    std::string client;  
};
```

```
ds::istream & operator >> ( ds::istream & in, job_type_t & type )  
{  
    char char_type;  
    in >> char_type;  
    type = static_cast< job_type_t >( char_type );  
    return in;  
}
```

```
ds::istream & operator >> ( ds::istream & in, job_key_t & key )  
{  
    return in >> key.id  
        >> key.type;  
}
```

```
ds::istream & operator >> ( ds::istream & in, job_t & job )  
{  
    return in >> job.key  
        >> job.client;  
}
```

```
ds::ostream & operator << ( ds::ostream & out, const job_type_t & type )  
{  
    return out << static_cast< char >( type );  
}
```

```
ds::ostream & operator << ( ds::ostream & out, const job_key_t & key )  
{  
    return out << key.id  
        << key.type;  
}
```

```
ds::ostream & operator << ( ds::ostream & out, const job_t & job )  
{  
    return out << job.key  
        << job.client;  
}
```


Custom and Aggregate type integration – usage example

```
ds::ostream out;  
job_t      job1 = { . . . };
```

```
out << job1 << ds::endr;
```

```
ds::istream in;  
job_t      job2;
```

```
in >> job2 >> ds::endr;
```

```
assert( job1 == job2 );
```

RDBMS Abstraction Layer

› C++ Classes

- Connection / Context
- Statement
- Result

› Pluggable RDBMS modules

- Currently supported types: sqlite, firebird, mysql, mssql
- Selectable by connection string – URI syntax: `type://location:port/path?key=value...#database`
- Consistent behaviour between modules
- Consistent data typing between modules eg: ISO-8601 (date / time)
- Named parameter support for all RDBMS types
- Isolation of encoding concerns whilst maintaining a consistent interface

› Pluggable SQL modules

- Application specific named SQL queries
- SQL selected by RDBMS connection type

RDBMS Abstraction Layer Classes – namespace ds::db

```
class connection : public impl
{
public:

    virtual statement operator()( const std::string & query,
                                   const name_list_t & parameters = {} ) override;

    virtual void execute_batch( const std::string & query ) override;
};
```

```
class statement : public ds::ostream
{
public:

    void execute( void );
    db::result result( void );
};
```

```
class result : public ds::istream
{
};
```

SQL Statement Cases

➤ Case One

- No parameters
- No result set
- Eg: Data definition language statements

➤ Case Two

- One or more parameters
- No result set
- Eg: Insert / Update / Delete statements

➤ Case Three

- Zero or more parameters
- Produces result set
- Eg: Select / Insert statements

Simple Usage Examples

- Case One – no parameters, no result set

```
connection.execute_batch( "CREATE TABLE job( . . . )" );
```

- Case Two – one or more parameters, no result set

```
statement = connection( "INSERT INTO job VALUES( ?, ?, ? )" );  
statement << id << type << client;  
statement.execute();
```

- Case Three – zero or more parameters, produces result set

```
statement = connection( "SELECT job_client FROM job WHERE . . ." );  
statement << id << type;  
result = statement.result();  
result >> client;
```

Reduced Syntax Usage Examples

- Case One – no parameters, no result set

```
connection.execute_batch( "CREATE TABLE job( . . . )" );
```

- Case Two – one or more parameters, no result set

```
statement = connection( "INSERT INTO job VALUES( ?, ?, ? )" );  
statement << job << endr; or  
statement( job ).execute(); or  
statement( id, type, client ).execute();
```

- Case Three – zero or more parameters, produces result set

```
statement = connection( "SELECT job_client FROM job WHERE . . ." );  
client = statement( job_key ); or  
client = statement( id, type );
```

STL Integration and Reduced Syntax Definitions

```
class statement : public ds::ostream
{
public:
    class iterator
    {
    };

    iterator begin( void );
    iterator end( void );

    template< typename T > statement & operator()( const T & t )
    {
        *this << t;
        return *this;
    }

    template< typename T, typename ... Rest >
    statement & operator()( const T & t, const Rest & ... rest )
    {
        *this << t;
        return operator()( rest... );
    }

    template< typename T > operator T ( void )
    {
        return result();
    }
};

template<> class std::iterator_traits< ds::db::statement::iterator >
{
public:
    typedef int32_t          difference_type;
    typedef ds::db::result    value_type;
    typedef value_type *      pointer;
    typedef value_type &      reference;
    typedef forward_iterator_tag iterator_category;
};
```

```
class result : public ds::istream
{
public:
    template< typename T > operator T ( void )
    {
        T t;
        *this >> t;
        return t;
    }
};
```

STL Integration Usage Example

```
ds::db::connection con("firebird://localhost:3050/C:/tmp#pmdb" );  
  
for ( job_t job : con( "SELECT FIRST 5 job_id, job_type, job_client FROM jobs" ) )  
    do_something( job );
```


Transactions and Savepoints

- Transactions and Savepoints control isolation, commit and rollback of DB changes
- All Datenstrom RDBMS encoders implement implied transactions consistently
- Transactions and Savepoints can also be declared explicitly – and should be
- `ds::db::transaction` and `ds::db::savepoint` classes are scope bound

Transaction Example

```
ds::db::connection con( "firebird://localhost:3050/C:/tmp#pmdb" );

job_t jobs[] = { { . . . }, { . . . }, . . . };

ds::db::namelist_t params = { "id", "type", "client" };

ds::db::statement insert = con( "INSERT INTO jobs VALUES( :id, :type, :client )", params );

ds::db::transaction txn( con );

for ( const job_t & job : jobs )
    insert << job << ds::endr;

// End of scope – commit transaction
```

Full Data Access Abstraction – The SQL Problem

- › context combines a connection with RDBMS specific SQL

```
class context : public connection
{
public:
    virtual void execute_batch( const std::string & query ) override;
    virtual statement operator()(const std::string & query, . . . ) override;
};
```

- › context usage with named SQL statement

```
ds::db::context ctx( connection_string );

for ( job_t job : ctx( "job.first_5" ) )
    do_something( job );
```

- › Connection strings can then be defined at runtime

```
firebird://localhost:3050/C:/tmp#pmdb
mssql://127.0.0.1#pmdb
mssql://analytics.database.windows.net#pmdb
sqlite:///C:/tmp#pmdb
```

RDBMS SQL modules

- SQL modules provide tables of named SQL queries categorised by RDBMS type
- Type: firebird

Name	Query (firebird specific)
job.first_5	SELECT FIRST 5 job_id, job_type, job_client FROM jobs
job.insert	INSERT INTO jobs . . .
. . .	

- Type: mssql

Name	Query (mssql specific)
job.first_5	SELECT job_id, job_type, job_client FROM jobs LIMIT 5
job.insert	INSERT INTO jobs . . .
. . .	

Integration Testing – gtest test suites

```

TEST_P( Job, should_provide_job_list )
{
    dm::job::job_vec_t source_vec =
    {
        { . . . },
        . . .
    };

    ds::db::context ctx( GetParam() );

    db_dal::Job_insert job_insert( ctx );
    db_dal::Job_list    job_list( ctx );

    for ( auto & job : source_vec )
        job_insert << job << std::endlr;

    dm::job::job_vec_t result_vec;
    result_vec = job_list.result();

    EXPECT_EQ( result_vec, source_vec );
}

TEST_P( Job, should_provide_job_detail )
{
    . . .
}

. . .

```

[illegible]